03b — Asynchronous Composition

Syntax

```
asynchronous name {
    members [A, B, C];
}
```

Declares an asynchronous composition of automata `A`, `B`, and `C` under the identifier `name`. All listed members must be defined in the same context.

Semantics

- Any component can take a step *independently* of the others.
- Components whose label is not involved remain in their current state (interleaving).
- Labels that appear in the transition sets of **multiple** members *must synchronize* — all participating automata step together.
- No `skip` / idle labels are required (unlike synchronous composition).
- The result is a single CLTS whose states are tuples of member states.

Sync vs Async — Comparison

Aspect	Synchronous	Asynchronous
Stepping	Lock-step (all advance)	Interleaved (one advances)
`skip` labels	Required for idle members	Not needed
Shared labels	Always sync	Sync only when shared
State space	Product of states	Full product of states
Use case	Tightly coupled	Loosely coupled

When to Use Async

- Independent subsystems (e.g. multiple sensors).
- Components with different clock domains.
- Event-driven systems where components react to stimuli at their own pace.
- When you want to avoid the overhead of explicit `skip` transitions in every automaton.

Shared Labels

A label is **shared** if it appears in the transition sets of two or more member automata.

- Shared labels act as *synchronization barriers*: all automata that know the label must take it simultaneously.
- Use **distinct** label names for actions that should remain independent.
- Shared labels are the primary mechanism for inter-component communication in async mode.

Example: if `A` and `B` both have label `sync_ab`, they must fire it together. `C` is unaffected and may step independently.

State Space

The composite state space is the Cartesian product of all member state sets:

$$|S| = |S_A| \times |S_B| \times |S_C| \times \cdots$$

Example: three automata with 2, 2, and 3 states yield $2 \times 2 \times 3 = 12$ composite states.

- Async typically produces more reachable transitions than sync (more interleavings).
- Use property-based reduction or minimization to manage large state spaces.